

CSP-J/S模拟(三)

2026年7月26日

说明： J 组不需要做附加题， S 组需要做 J 的所有题目及附加题。

一、单项选择题

- 共 15 题，每题 2 分，共计 30 分
- 每题有且仅有一个正确选项

1. $(2025)_8 + (2025)_{16}$ 和以下哪个选项相等（ ）。
A. $(9244)_{10}$
B. $(2100222)_4$
C. $(10010000111010)_2$
D. $(234A)_{16}$
2. 以下（ ）函数声明是合法的。
A. `int Bubblesort(char a[][],int n)`
B. `int Bubblesort(char a[10][],int n)`
C. `int Bubblesort(char a[][20],int n)`
D. `int Bubblesort(char [,] a,int n)`
3. 设循环队列中数组的下标范围是 $0 \sim n - 1$ ，其头尾指针分别为 f 和 r ，则其元素个数为（ ）。
A. $r - f$
B. $r - f + 1$
C. $(r - f) \% n + 1$
D. $(r - f + n) \% n$
4. 后缀表达式 `1 2 + 3 * 14 7 / -` 对应的前缀表达式为（ ）。
A. `1 + 2 * 3 - 14 / 7`
B. `- * 1 + 2 3 / 14 7`
C. `- * + 1 2 3 / 14 7`
D. `- 1 + 2 * 3 14 / 7`
5. 给定一棵二叉树，其前序遍历结果为 $abdecfg$ ，中序遍历结果为 $debacfg$ ，则这棵树的后序遍历结果为（ ）。
A. $edbgfca$
B. $edgbfca$
C. $debgfca$
D. $dbegfca$
6. 下列是关于数据结构的说法不正确的是（ ）。
A. 数据结构是带有结构的数据元素的集合
B. 线性表的线性存储结构优于链式存储结构
C. 队列是一个先进先出的线性表
D. 队列是只能在一端插入，另一端删除的线性表

7. 已知无向图 G 含有16条边,其中度为4的顶点个数为3,度为3的顶点个数为4,其他顶点的度均小于3。 G 所含的顶点个数至少是 ()。
- A. 10
B. 11
C. 13
D. 15
8. 下面关于指针的说法正确的是 ()。
- A. 在64位计算机中一个指针变量占4字节
B. 指针运算实际上是地址操作、只能取地址和间接访问,不能进行加减运算
C. 数组名不是指向数组元素的指针变量
D. 指针只可以静态申请内存空间
9. 方程 $a \times b = (a \text{ or } b) \times (a \text{ and } b)$, 在 a, b 都取 $[0, 31]$ 中的整数时, 共有 () 组解。($\times$ 表示乘法; **or** 表示按位或运算; **and** 表示按位与运算)
- A. 32
B. 256
C. 454
D. 512
10. 双向链表中有两个指针域, $llink$ 和 $rlink$, 分别指回前驱及后继, 设 p 指向链表中的一个结点, q 指向一待插入结点, 现要求在 p 前插入 q , 则正确的插入为 ()。
- A. `p->llink = q; q->rlink = p; p->llink->rlink = q; q->llink = p->llink;`
B. `q->llink = p->llink; p->llink->rlink = q; q->rlink = p; p->llink = q->rlink;`
C. `p->llink->rlink = q; q->rlink = p; q->llink = p->llink; p->llink = q;`
D. `q->rlink = p; p->rlink = q; p->llink->rlink = q; q->rlink = p;`
11. 将2, 6, 10, 17分别存储到某个地址区间为0 — 10的哈希表中, 如果哈希函数 $h(x) = ()$ 将不会产生冲突, 其中 $a \bmod b$ 表示 a 除以 b 的余数。
- A. $x \bmod 11$
B. $x^2 \bmod 11$
C. $(2x) \bmod 11$
D. $\lfloor \sqrt{x} \rfloor \bmod 11$
12. 输入时 n 个不等的数构成的数组 a , 输出 a 中第二小的数。在最坏的情况下, 该算法需要做 () 次比较。

```

1      if (a[1] < a[2])
2      {
3          min1 = a[1];
4          min2 = a[2];
5      }
6      else
7      {
8          min1 = a[2];
9          min2 = a[1];
10     }
11     for(int i = 3; i <= n; i++)
12         if (a[i] < min2)
13             if (a[i] < min1)
14                 {

```

```

15         min2 = min1;
16         min1 = a[i];
17     }
18     else
19     {
20         min2 = a[i];
21     }

```

- A. $2n - 1$
- B. $2n - 2$
- C. $2n - 3$
- D. $2n$

13. 整型数组 `a` 中有 `n` 个元素，能计算 `a` 中有多少个数字大于 `lower` 且小于 `upper` 的函数，应该将下划线依次替换为（ ）。

```

1  int solve(int a[], int n, int lower, int upper)
2  {
3      std::sort(a, a + n);
4      auto begin = std::_____(a, a + n, lower);
5      auto end = std::_____(a, a + n, upper);
6      return end - begin;
7  }
8

```

- A. `lower_bound`、`lower_bound`
- B. `lower_bound`、`upper_bound`
- C. `upper_bound`、`lower_bound`
- D. `upper_bound`、`upper_bound`

14. 若 $f_0 = 0, f_1 = 1, f_{n+1} = \frac{f_n + f_{n-1}}{2}$ ，则随着 i 的增大， f_i 将接近于（ ）。

- A. $\frac{1}{2}$
- B. $\frac{2}{3}$
- C. $\frac{\sqrt{5}-1}{2}$
- D. 1

15. 由四个没有区别的点构成的简单无向连通图的个数是（ ）。

- A. 6
- B. 7
- C. 8
- D. 9

二、阅读程序

- 判断题 1 分，选择题 3 分，共计 40 分
- 判断题正确填 `T`，错误填 `F`；

第1题

```
1  using i64 = long long;
2  i64 f(i64 n)
3  {
4      i64 s = 1;
5      i64 i = 2;
6      while (i * i < n)
7      {
8          if (n % i == 0) {
9              s += i;
10             s += n / i;
11         }
12         ++i;
13     }
14     if (i * i == n) s += i;
15     return s;
16 }
```

判断题

16. 当 $n = 1$ 时, 函数返回 1 ()。
17. 当 n 是质数时, 函数返回 n ()。
18. 存在两个数 $n \neq m$ 且 $f(n) = f(m)$ ()。
19. 程序的时间复杂度为 $\Theta(\log n)$ ()。

选择题

20. `f(100)` 会进入 `while` 循环次数是 ()。
- A. 7
- B. 8
- C. 9
- D. 10
21. 运行 `f(1024)` 时, 返回值是 ()。
- A. 512
- B. 1023
- C. 1024
- D. 2047

第2题

```
1  int solve(int n, int a[])
2  {
3      int ret = 0;
4      for (int i = 0; i < n; ++i)
5      {
6          for (int j = 0; j < i; ++j)
7          {
```

```

8         int sum = 0;
9         for (int k = j; k <= i; ++k)
10        {
11            sum += a[k];
12        }
13        ret += sum;
14    }
15    }
16    return ret;
17 }

```

判断题

22. 若在进入 `solve` 函数执行其他操作之前，先对 `a[]` 排序，返回值不变（ ）。
23. 当 `n = 1` 时，函数返回值为 `a[0]`（ ）。
24. 若数组 `a[]` 中所有元素均为0，则当 `n` 取1到100之间的整数时，函数一定返回0（ ）。

选择题

25. 若 `n = 10` 且 $a = \{1, 1, \dots, 1, 1\}$ ，程序的返回值是（ ）。
- A. 10
B. 100
C. 165
D. 210
26. 若 `n = 5`，且 $a = \{3, 1, 4, 1, 5\}$ ，则程序返回（ ）。
- A. 14
B. 75
C. 78
D. 80
27. 该程序的时间复杂度为（ ）。
- A. $\Theta(n)$
B. $\Theta(n^2)$
C. $\Theta(n^2 \cdot \log n)$
D. $\Theta(n^3)$
28. 如果打算用更好的算法实现 `solve` 函数，那么最好的算法可以达到的时间复杂度为（ ）。
- A. $\Theta(n)$
B. $\Theta(n^2)$
C. $\Theta(n^2 \cdot \log n)$
D. $\Theta(\log n)$

第3题

```

1  using i64 = long long;
2
3  i64 solve1(i64 n)
4  {
5      std::vector<i64> c(n);

```

```

6      c[0] = 0;
7      i64 sum = 0;
8      for (i64 i = 1; i < n; ++i)
9      {
10         c[i] = c[i / 2] + (i % 2);
11         sum += c[i];
12     }
13     return sum;
14 }
15
16 std::pair<i64,i64> solve2(i64 n)
17 {
18     if (n == 0)
19         return {0, 0};
20     auto r = n % 2;
21     auto q = n / 2;
22     auto [s, c] = solve2(q);
23     if (r == 1)
24         return {s*2 + q + c, c + 1};
25     else
26         return {s*2 + q, c};
27 }

```

判断题

29. `solve1(5)` 返回 5 ()。
30. `solve2(8)` 返回 {10, 1} ()。
31. 若输入参数 n 在 0 到 1024 之间, 则 `solve1(n)` 的返回值与 `solve2(n)` 的第一项返回值必定相等 ()。

选择题

32. `solve1(n)` 计算的是 ()。
- 0 到 $n - 1$ 之间, 全体二进制数的零出现的数量
 - 0 到 $n - 1$ 之间, 全体二进制数的一出现的数量
 - 0 到 n 之间, 全体二进制数的零出现的数量
 - 0 到 n 之间, 全体二进制数的一出现的数量
33. `solve2(n)` 的第二个返回值, 计算的是 ()。
- 参数 `n` 的二进制长度
 - 参数 `n` 的十进制长度
 - 参数 `n` 的在二进制表示下, 0 的数量
 - 参数 `n` 的在二进制表示下, 1 的数量
34. `solve1` 及 `solve2` 的时间复杂度是 ()。
- $\Theta(n)$ 、 $\Theta(n)$
 - $\Theta(n)$ 、 $\Theta(\log n)$
 - $\Theta(\log n)$ 、 $\Theta(n)$
 - $\Theta(\log n)$ 、 $\Theta(\log n)$
35. `solve1(4096)` 的返回值等于 ()。
- 4096×5

- B. 4096×6
- C. 2048×5
- D. 2048×6

三、完善程序

- 单选题，每小题3分，共计30分

第1题

两人进行 N 次石头剪刀布游戏，给定对方的出拳序列，由 R 、 P 、 S 组成（分别表示石头、剪刀、布）。你的出拳需满足：

- 从未输过（每次非赢即平）。
- 相邻两次出拳不同。

求可能赢的最大对局数（即赢的次数，平局不计入）。

```
1  #include<iostream>
2  int score(int a,int b){
3      if(____(1)____) return 0;
4      else if(a=='R' and b=='S')return 1;
5      else if(____(2)____)return 1;
6      else if(a=='P' and b=='R')return 1;
7      else return -10000000;
8  }
9  int main(){
10     ____ (3) ____
11     int n;
12     std::cin>>n;
13     while(n-->0){
14         char c;
15         std::cin>>c;
16         int newR = ____ (4) ____;
17         int newS = ____ (5) ____;
18         int newP = ____ (6) ____;
19         R=newR;
20         S=newS;
21         P=newP;
22     }
23     std::cout<< ____ (7) ____;
24 }
```

36. (1) 处应填（ ）。

- A. `a != b`
- B. `a < b`
- C. `a > b`

- D. `a == b`
37. (2) 处应填 ()。
- A. `a=='S' and b=='P'`
- B. `a=='S' and b=='R'`
- C. `a=='P' and b=='S'`
- D. `a=='R' and b=='P'`
38. (3) 处应填 ()。
- A. `char R,S,P`
- B. `char R = 'R',S = 'S',P = 'P'`
- C. `int R = 0,S = 0,P = 0`
- D. `int R = 'R',S = 'S',P = 'P'`
39. (4)(5)(6) 处应填 ()。
- A. `score(R,c)`、`score(S,c)`、`score(P,c)`
- B. `score('R',c)`、`score('S',c)`、`score('P',c)`
- C. `R + score('R',c)`、`S + score('S',c)`、`P + score('P',c)`
- D. `std::max(S,P)+score('R',c)`、`std::max(R,P)+score('S',c)`、`std::max(R,S)+score('P',c)`
40. (7) 处应填 ()。
- A. `std::min(std::min(R,S),P)`
- B. `std::max(std::max(R,S),P)`
- C. `n - std::min(std::min(R,S),P)`
- D. `n - std::max(std::max(R,S),P)`

第2题

给定 n 根火柴的长度 $a_1, a_2, \dots, a_n$ ，请用这些火柴围成一个面积最大的三角形。注意所有的火柴都必须用上，不得丢弃。输出最大三角形的面积。假设最大面积为 s ，则输出 $16s^2$ 。 $1 \leq a_i \leq 40$ ，数据保证至少有一种方案可以围成三角形。

```

1  #include<iostream>
2  using i64 = long long;
3  int n;
4  int a[40];
5  bool mem[40][40*40][40*40];
6  i64 value[40][40*40][40*40];
7
8  i64 solve(int i, int x, int y, int z) {
9      if (i < n) {
10         if (mem ____ (1) ____ > 0) return value ____ (2) ____;
11         auto s1 = solve(i+1, x + a[i], y, z);
12         auto s2 = solve(i+1, x, y + a[i], z);
13         auto s3 = solve(i+1, x, y, z + a[i]);
14         mem ____ (3) ____ = true;
15         return value ____ (4) ____ = ____ (5) ____;
16     }
17     else {
18         if ( ____ (6) ____ ) return 0;
19         i64 p = ____ (7) ____ ;

```



```

20         return ____ (8) ____ ;
21     }
22 }
23
24 int main()
25 {
26     std::cin >> n;
27     for (int i = 0; i < n; ++i) {
28         std::cin >> a[i];
29     }
30     std::cout << solve(0, 0, 0, 0);
31 }

```

41. (1)、(2)、(3)、(4)处应填 () 。

- A. `[i][x][y],[i][x][y],[i][x][y],[i][x][y]`
- B. `[i][x][y],[x][y][z],[i][x][y],[x][y][z]`
- C. `[x][y][z],[i][x][y],[x][y][z],[i][x][y]`
- D. `[x][y][z],[x][y][z],[x][y][z],[x][y][z]`

42. (5) 处应填 () 。

- A. `s1 + s2 + s3`
- B. `*std::max_element({s1, s2, s3}.begin(), {s1, s2, s3}.end())`
- C. `std::max(std::max(s1, s2), std::max(s3, z))`
- D. `std::max(std::max(s1, s2), s3)`

43. (6)处应填 () 。

- A. `x + y <= z && x + z <= y && y + z <= x`
- B. `x + y <= z || x + z <= y || y + z <= x`
- C. `x + y > z || x + z > y || y + z > x`
- D. `x + y >= z && x + z >= y && y + z >= x`

44. (7)处应填 () 。

- A. `x * y * z`
- B. `x + y + z`
- C. `(x * y) / 2`
- D. `(x + y + z) / 2`

45. (8)处应填 () 。

- A. `p * (p - 2 * x) * (p - 2 * y)`
- B. `(p / 2) * (p / 2 - x) * (p / 2 - y) * (p / 2 - z)`
- C. `p * (p - 2 * x) * (p - 2 * y) * (p - 2 * z)`
- D. `p + (p - 2 * x) + (p - 2 * y) + (p - 2 * z)`

四、附加题

- 判断题 1 分，选择题 3 分，共计 30 分
- 判断题正确填 **T**，错误填 **F**；

第1题

```
1  int exp();
2
3  int term()
4  {
5      char begin, end;
6      std::cin >> begin;
7      int t = exp();
8      std::cin >> end;
9      if (t == 0)
10         return 1;
11     else
12         return t * 2;
13 }
14
15 int exp()
16 {
17     if (std::cin.peek() == '(')
18     {
19         return term() + exp();
20     }
21     else
22     {
23         return 0;
24     }
25 }
26
27 int main()
28 {
29     std::cout << exp() << "\n";
30 }
```

判断题

46. 输入 `((()))()`，程序运行结果输出 `3`。 ()
47. 输入 `((()))` 输出 `3`。 ()
48. 输出 `15` 的最短输入序列之一是 `((((( )))())())`。 ()

选择题

49. 若输入是 `((()))()()()()()`，输出 ()。
- A. 16
- B. 18
- C. 20
- D. 22
50. 以下哪个输入对应的输出为 `10` ()。
- A. `((()))()`
- B. `((()))()()()`

C. `((()))()((()))()`

D. `((()))()((()))()`

51. 若输入字符串 `((()))()`，代码执行过程中 `term()` 函数被调用的次数是（ ）。

A. 4

B. 5

C. 7

D. 8

52. 若输入 `((()))()()`，则 `exp()` 函数总共被调用的次数是（ ）。

A. 7

B. 9

C. 13

D. 15

第2题

某个投资者有 m 元钱，有 n 个项目等待他的投资，每个项目只能投资一次。

代码中的一个 `pair` 表示一个项目。其中第 i 个项目要求先支出成本 c_i 元，待项目完成后，可以收回全部成本，且获得 p_i 元利润，若投资者的钱不足 c_i ，就没法投资这个项目了。

可以用老项目收回的成本及利润支付新项目的成本。若只能投资 k 个项目，那么投资者最终可以积累多少钱呢。

```
1 struct pair
2 {
3     int c;
4     int p;
5 };
6
7 int solve(int n, int k, int m, pair a[])
8 {
9     auto compare = [] (pair first, pair second) {
10         return ____ (1) ____;
11     };
12     std::sort(a, a + n, compare);
13     std::priority_queue<int> Q;
14     int size = 0;
15     while (k > 0) {
16         while (____ (2) ____ ) {
17             Q.push(____ (3) ____);
18         }
19         if (____ (4) ____ ) {
20             ____ (5) ____;
21         }
22         k--;
23     }
24     return m;
25 }
```

53. (1)处应填 ()。

- A. `first.c < second.c`
- B. `first.c > second.c`
- C. `first.p < second.p`
- D. `first.p > second.p`

54. (2) 处应填 ()。

- A. `size < n && a[size].c <= m`
- B. `size < n && a[size].c < m`
- C. `size <= n && a[size].c <= m`
- D. `size < n || a[size].c <= m`

55. (3)处应填 ()。

- A. `a[size++].p`
- B. `a[++size].p`
- C. `a[size++].c`
- D. `a[++size].c`

56. (4) 处应填 ()。

- A. `!Q.empty()`
- B. `Q.empty()`
- C. `size > 0`
- D. `k > 0`

57. (5) 处应填 ()。

- A. `m += Q.top(); Q.pop();`
- B. `m += Q.front(); Q.pop();`
- C. `m += Q.front().p - Q.front().c;`
- D. `Q.pop(); m += Q.top();`